

Relatório

- Crítica

- Falta de tratamento de entradas inválidas – Percebi no meu código que ele utiliza apenas `scanf` para entrada de dados, então se em um `scanf` que pede uma entrada do tipo `%d` (decimal), e eu insiro uma string, meu código vai quebrar ali mesmo. O que faz com que qualquer erro de digitação quebre o programa.
- Quando se trata de memória alocada de maneira estática, que é o caso dos vetores, eles trabalham em uma parte da memória chamada Stack que tem um limite de memória bem reduzido, geralmente de 1mb ou 2mb, no entanto, para processos grandes, como uma fila ou pilha de pacientes muito grande, ele vai causar um problema chamado Stack overflow, que acontece por tentar reservar um espaço de memória maior do que o limite disponível logo na inicialização do programa.

- Bugs

BUGS - VERSÃO SEM PONTEIROS.

```
main.c:9:5: warning: scanf() without field width limits can crash with huge input data. [invalidscanf]
scanf("%[^\n]", p.nome);
^
```

Foi identificado um único warning relacionado à função `scanf`. Este comando realiza a leitura para um vetor de char que suporta até 100 caracteres. Caso o usuário insira uma quantidade maior, o programa tentará alocar todos esses caracteres além do espaço de memória reservado para o vetor. Esse comportamento causa um estouro de buffer (buffer overflow), invadindo o espaço de variáveis vizinhas e provocando o fechamento forçado (*crash*) do programa.

```
4/4 files checked 100% done
consultas.c:12:5: style: The function 'estaCheiaConsultas' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int estaCheiaConsultas(consulta c){
^
consultas.c:16:5: style: The function 'estaVaziaConsultas' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int estaVaziaConsultas(consulta c){
^
emergencia.c:11:5: style: The function 'EstaCheia' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int EstaCheia(Emergencia p){
^
emergencia.c:15:5: style: The function 'EstaVazia' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int EstaVazia(Emergencia p){
^
exames.c:12:5: style: The function 'estaCheiaExames' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int estaCheiaExames(Exame e){
^
exames.c:16:5: style: The function 'estaVaziaExames' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int estaVaziaExames(Exame e){
^
main.c:6:10: style: The function 'CadastrarPaciente' should have static linkage since it is not used outside of its translation unit. [staticFunction]
Paciente CadastrarPaciente(){
^
}
```

Para este caso, trata-se de alertas do tipo **Style** (Estilo), ou seja, inconformidades que não impedem a compilação do código. A ferramenta identificou funções auxiliares internas que são invocadas apenas dentro do próprio arquivo onde foram declaradas. Como boa prática de desenvolvimento, o Cppcheck recomenda defini-las com a diretiva static (**static linkage**). Isso restringe o escopo dessas funções ao seu próprio módulo, melhorando o encapsulamento e permitindo que o compilador realize otimizações de memória.

BUGS - VERSÃO COM PONTEIROS.

```
exames_p.c:11:9: note: Null pointer dereference
e->pacientes = (Paciente*) malloc(e->tamanho* sizeof(Paciente));
^
exames_p.c:11:43: warning: If memory allocation fails, then there is a possible null pointer dereference: e [nullPointerOutOfMemory]
e->pacientes = (Paciente*) malloc(e->tamanho* sizeof(Paciente));
^
exames_p.c:5:37: note: Assuming allocation function fails
Exames *e = (Exames*) malloc(sizeof(Exames));
^
exames_p.c:5:21: note: Assignment 'e=(struct Exames*)malloc(sizeof(struct Exames))', assigned value is 0
Exames *e = (Exames*) malloc(sizeof(Exames));
^
exames_p.c:11:43: note: Null pointer dereference
e->pacientes = (Paciente*) malloc(e->tamanho* sizeof(Paciente));
^
exames_p.c:16:29: style: Parameter 'e' can be declared as pointer to const [constParameterPointer]
int estaChelaExames(Exames *e){
^
exames_p.c:20:29: style: Parameter 'e' can be declared as pointer to const [constParameterPointer]
int estaVaiasExames(Exames *e){
^
3/4 files checked 48% done
Checking main_p.c ...
main_p.c:12:2: information: Include file: <stdio.h> not found. Please note: Standard library headers do not need to be provided to get proper results. [missingIncludeSystem]
#include <stdio.h>
^
main_p.c:2:2: information: Include file: <stdlib.h> not found. Please note: Standard library headers do not need to be provided to get proper results. [missingIncludeSystem]
#include <stdlib.h>
^
main_p.c:7:2: information: Include file: <time.h> not found. Please note: Standard library headers do not need to be provided to get proper results. [missingIncludeSystem]
#include <time.h>
^
4/4 files checked 100% done
consultas_p.c:17:5: style: The function 'estaChelaConsultas' should have static linkage since it is not used outside of its translation unit. [staticFunction]
int estaChelaConsultas(Consultas *c){
^
```

A análise estática realizada via Cppcheck na versão dinâmica concluiu 100% da varredura dos arquivos fontes, apontando alertas preventivos focados em segurança de memória na Heap. O relatório destacou avisos do tipo **nullPointerOutOfMemory** nas funções de inicialização (malloc), simulando cenários hipotéticos onde a falta de memória RAM faria a função retornar um ponteiro nulo (NULL). Adicionalmente, a ferramenta indicou sugestões de melhorias estilísticas de escopo, como a declaração de parâmetros constantes **constParameterPointer** em funções de checagem interna e uso de ligação estática (staticFunction). Tais apontamentos foram utilizados como guia de boas práticas para blindar a integridade dos ponteiros do sistema.

- Análise de estresse e tempo

ANÁLISE – SEM PONTEIROS

```
--- RESULTADOS DO TESTE DE ESTRESSE(EM SEGUNDOS) ---  
Setor Emergencia(Pilha) | Insercao: 0.002000 segundos | Atendimento: 0.002000 segundos  
Setor Consultas(Fila)   | Insercao: 0.001000 segundos | Atendimento: 0.002000 segundos  
Setor Exames(Fila Circular) | Insercao: 0.001000 segundos | Atendimento: 0.002000 segundos
```

Podemos perceber neste caso que a inserção e remoção em todos os casos de Pilha/Fila, foram de tempos praticamente identicos, e isso ocorre porque estamos utilizando vetores estáticos, então inserir ou remover um elemento significa simplesmente colocar o paciente na posição do índice atual($\text{vetor}[\text{topo}] = p$, na Pilha, $\text{vetor}[\text{fim}] = p$, na Fila) ou remover do índice atual($\text{vetor}[\text{topo}] = p$, para Pilha, $\text{vetor}[\text{frente}] = p$, para Fila). E isso é um comportamento de complexidade $O(1)$.

ANÁLISE – COM PONTEIROS

```
--- INICIANDO TESTE DE ESTRESSE E TEMPO ---  
--- SETOR DE EMERGÊNCIA (PILHA DINÂMICA) ---  
-> Tempo para Empilhar 10.000: 0.254000 segundos.  
-> Tempo para Desempilhar 10.000: 0.276000 segundos.  
  
--- SETOR DE CONSULTAS (FILA LINEAR) ---  
-> Tempo para Enfileirar 10.000: 0.249000 segundos.  
-> Tempo para Desenfileirar 10.000: 0.282000 segundos.  
  
--- SETOR DE EXAMES (FILA CIRCULAR) ---  
-> Tempo para Enfileirar 10.000: 0.282000 segundos.  
-> Tempo para Desenfileirar 10.000: 0.294000 segundos.
```

Assim como nos sem ponteiros, aqui os tempos foram muito parecidos também, pois a complexidade continua sendo $O(1)$, e os ponteiros chegam e trazem um diferencial, de poder alocar dinamicamente a quantidade de memória necessária para o processo, isso evita desperdícios de memória.

CONCLUSÃO

Ao realizar esta sequência de pesquisas, implementações e testes, observou-se que a versão com ponteiros é a que melhor se adapta a cenários reais. Por utilizar alocação dinâmica, ela evita desperdícios de memória e suporta um volume de dados muito maior do que a versão estática. Isso ocorre porque as estruturas dinâmicas atuam na memória Heap, diferentemente da abordagem estática que opera na Stack — cujo limite de capacidade física geral gira em torno de apenas 2 MB.